

CLAUDE CODE × SERVICENOW SDK

Video 4: Transform & Update

Pull an Existing App from Your Instance, Modify with AI, Redeploy

By Tushar | ReAcademy.ai | March 2026

Prerequisites

- Videos 1–3 completed
- An existing application on your ServiceNow instance (can be the one from Video 3)

What is Transform?

The transform command downloads metadata from your ServiceNow instance and converts supported types into Fluent TypeScript (.now.ts) files. This lets you take any existing app and start working on it locally with Claude Code.

Use this when:

- You have apps built in the platform that you want to move to source-driven development
- A team member made changes on the instance and you need to sync locally
- You want to use Claude Code to enhance an existing app

Step 1 — Set Up Your Project Folder

If you already have an SDK project for this app:

```
cd my-existing-app
```

If starting fresh:

```
mkdir my-app && cd my-app
```

```
now-sdk init --from <sys_id or path>
```

During init, select the same scope/app that exists on your instance.

Step 2 — Run Transform

```
now-sdk transform --from metadata/update --auth devuser1
```

What happens:

- SDK connects to your instance and downloads all metadata for the app scope
- Supported metadata types are converted to Fluent .now.ts files
- Files are placed in src/fluent/generated/

Check the output:

```
ls src/fluent/generated/
```

You'll see folders for tables, business rules, ACLs, and other metadata types that the SDK could convert.

NOTE: *Not all metadata types are supported by transform yet. Complex flows, UI pages, and some advanced configurations may not convert. Check the SDK documentation for the current list of supported types.*

Step 3 — Modify with Claude Code

3a. Launch Claude Code

```
claude
```

3b. Ask Claude to make changes

Example prompt:

```
Look at the existing table definition in src/fluent/generated/
and add a new field called Risk as a string
column. Also create a business rule that logs a warning
when a record is viewed
```

Claude reads the transformed files, understands the existing schema, and generates the changes using the correct scope prefix and Fluent syntax.

3c. Move edited files out of generated/

Best practice: once you start editing a transformed file, move it from generated/ to your own directory.

```
mkdir -p src/fluent/tables
```

```
mv src/fluent/generated/tables/it-assets.now.ts src/fluent/tables/
```

TIP: Keep generated/ as an unmodified baseline. If you need to re-transform later, you can compare against it. Your edited files in src/fluent/ take precedence during build.

Step 4 — Build & Redeploy

```
now-sdk build
```

If the build succeeds:

```
now-sdk install --auth mydev
```

Switch to your browser and verify the changes on your instance.

Step 5 — The Transform Workflow Loop

Here's the development cycle when working with existing apps:

Phase	What You Do
Pull	now-sdk transform --auth <alias> — download latest from instance
Edit	Launch claude, describe changes in natural language
Build	now-sdk build — compile to metadata
Deploy	now-sdk install --auth <alias> — push to instance
Repeat	If changes were made on instance, transform again to sync

WARNING: Be careful with the `--generate-deletes true` flag on install. This removes records from the instance that you deleted locally. Only use it intentionally.

Useful Flags

Command / Flag	Purpose
<code>now-sdk transform --auth <alias></code>	Pull metadata to local Fluent files
<code>now-sdk build --debug true</code>	Include debug info in build output
<code>now-sdk build --generate-deletes true</code>	Mark local deletions for removal on install
<code>now-sdk install --auth <alias></code>	Deploy to instance
<code>now-sdk dependencies --auth <alias></code>	Fetch type defs for IntelliSense